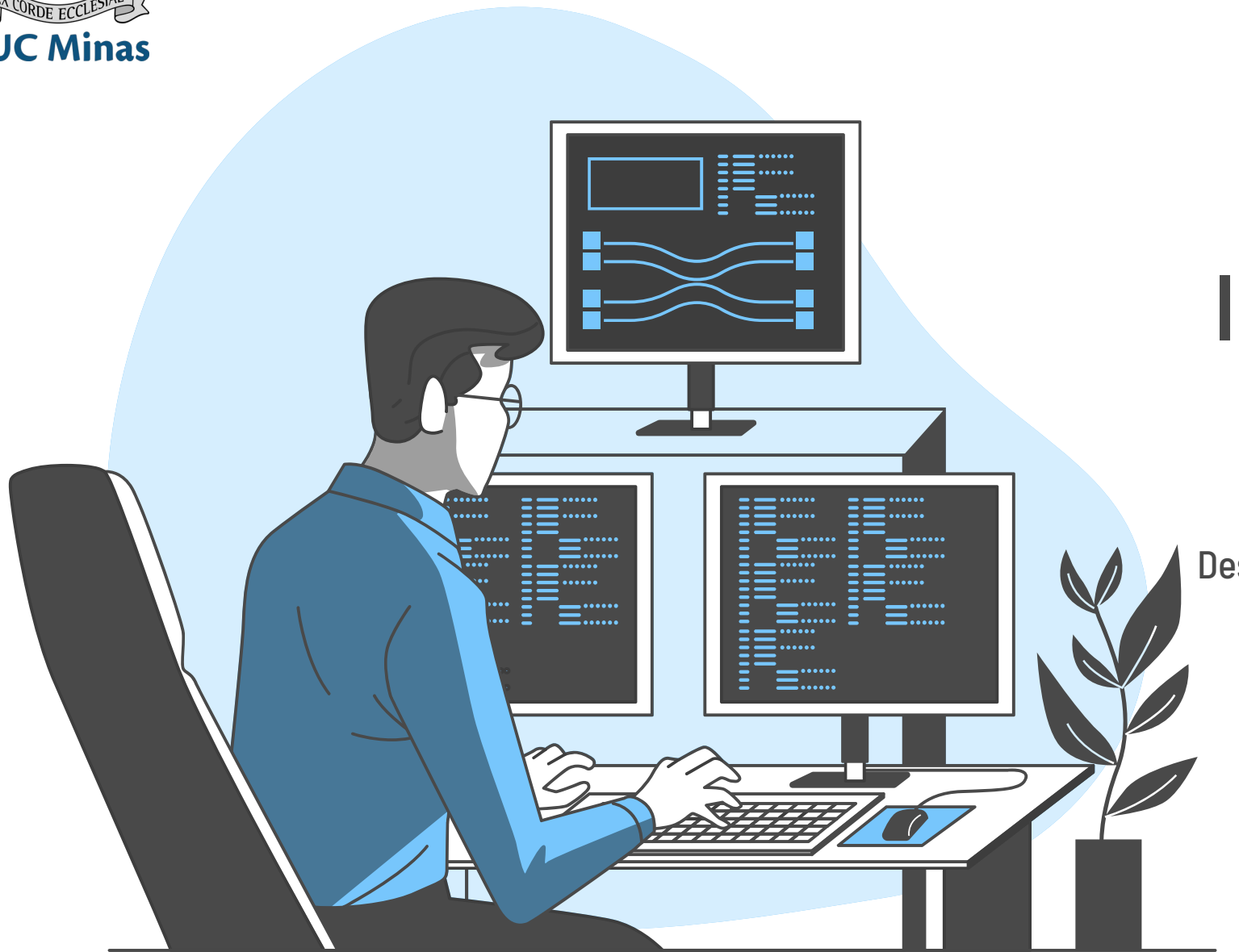
PUC Minas



Aula 07 · FullStack

Integrando a pasta Front dentro do Back

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Entender a pasta `src/main/resources/static` do Spring

2

Gerar o build do Vite **direto** no `static/` do back

3

Fazer o **fallback de SPA** (F5 em qualquer rota abre o React)

4

Automatizar com o **frontend-maven-plugin**: `./mvnw package` gera um `.jar` com tudo

5

Ajustar URL da API, CORS e Spring Security para mesma origem

Antes de começar

- Aulas 04 e 06 desta pasta
- Front-end, aula 03 (build do Vite)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 A ideia

02 Opção A: build do Vite direto no static do Spring

03 Opção B: o Maven faz tudo (frontend-maven-plugin)

04 Ajustes de segurança e deploy

05 Aprofundando: um Dockerfile para o projeto unificado

01

A ideia

A ideia

O Spring Boot serve automaticamente tudo que está em `src/main/resources/static/` (Back-end, aula 08). O resultado do `npm run build` é justamente uma pasta de arquivos estáticos. Se colocarmos o `dist/` dentro do `static/`, o próprio Spring entrega o React em `http://localhost:8080/` e a API em `http://localhost:8080/api/...`.

01

A ideia

Aula 07 · FullStack

A ideia



Um só servidor: arquivos do React e API na mesma origem. Sem CORS, um só deploy.

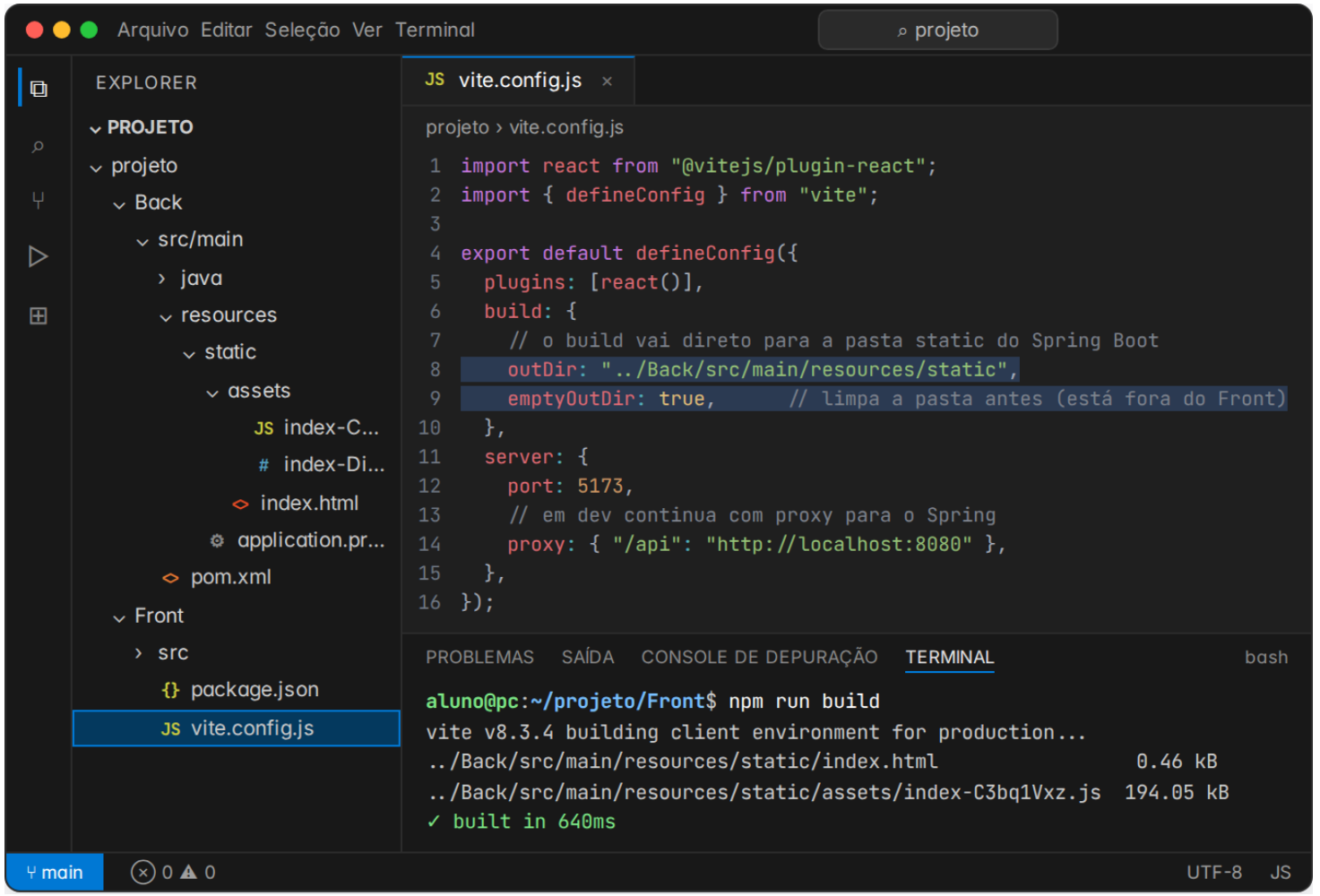
A ideia

	Separados (Vercel + Render)	Front dentro do Back
Deploys	2	1 (só o Render)
CORS	Necessário	Não existe (mesma origem)
`VITE_API_URL`	URL do Render	Vazia (URLs relativas)
Front fica	Na CDN do Vercel (rápido, sempre ligado)	No Render (dorme no plano gratuito)
Build	Independentes	O build do back depende do Node

02

Opção A: build do Vite direto no static do Spring

Opção A: build do Vite direto no static do Spring



Mantenha as pastas lado a lado e mude a saída do build do Vite:

`vite.config.js` com `outDir` apontando para o back. Depois do `npm run build`, os arquivos aparecem em `Back/.../static`.

Opção A: build do Vite direto no static do Spring

Atenção

`emptyOutDir: true` **apaga tudo** que estiver em `static/` antes do build. Se o back tinha arquivos próprios lá (CSS do Thymeleaf, imagens), mova-os para outro lugar ou use uma subpasta. Acrescente `Back/src/main/resources/static/` ao `.gitignore`: é conteúdo gerado.

Opção A: build do Vite direto no static do Spring

A URL da API passa a ser **relativa** (`/api/alunos`), porque o front e a API estão na mesma origem. Com o `apiConfig.js` da aula 06 (`VITE_API_URL ?? ""`), basta não definir `VITE_API_URL`.

02

Opção A: build do Vite direto no static do Spring

Aula 07 · FullStack

O fallback de SPA

Acesse `http://localhost:8080/alunos` e aperte F5: o Spring procura um arquivo ou endpoint `/alunos`, não acha e devolve **404**. Precisamos dizer: *"se não for API nem arquivo existente, entregue o `index.html` e deixe o React Router decidir"* .

Back/src/main/java/.../config/SpaConfig.java

```java
@Configuration
public class SpaConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/**")
                .addResourceLocations("classpath:/static/")
                .resourceChain(true)
                .addResolver(new PathResourceResolver() {
                    @Override
                    protected Resource getResource(String caminho, Resource local) throws IOException {
                        Resource arquivo = local.createRelative(caminho);
                        // arquivo existe (JS, CSS, imagem)? entrega. Senão, index.html
                        return arquivo.exists() && arquivo.isReadable()
                                ? arquivo
                                : new ClassPathResource("/static/index.html");
                    }
                });
    }
}
```

O fallback de SPA

Os controllers (`/api/**`) têm prioridade sobre esse *resource handler*, então a API continua funcionando normalmente.

02

Opção A: build do Vite direto no static do Spring

Aula 07 · FullStack

03

Opção B: o Maven faz tudo (frontend-maven-plugin)

Opção B: o Maven faz tudo (frontend-maven-plugin)

Na opção A alguém precisa lembrar de rodar `npm run build` antes do `./mvnw package`. O **frontend-maven-plugin** baixa um Node só para o build, roda `npm ci` e `npm run build` **durante o build do Maven**. Assim `./mvnw clean package` gera um `.jar` que já contém o React.

03

Opção B: o Maven faz tudo (frontend-maven-plugin)

Aula 07 · FullStack

Opção B: o Maven faz tudo (frontend-maven-plugin)

Back/pom.xml (dentro de <build><plugins>)

```
<plugin>
  <groupId>com.github.eirslett</groupId>
  <artifactId>frontend-maven-plugin</artifactId>
  <version>1.15.1</version>
  <configuration>
    <workingDirectory>../Front</workingDirectory>      <!-- onde está o package.json -->
    <installDirectory>target</installDirectory>        <!-- Node baixado fica em target/ -->
  </configuration>
  <executions>
    <execution>
      <id>instalar-node</id>
      <goals><goal>install-node-and-npm</goal></goals>
      <configuration><nodeVersion>v22.20.0</nodeVersion></configuration>
    </execution>
```

Opção B: o Maven faz tudo (frontend-maven-plugin) (cont.)

Back/pom.xml (dentro de <build><plugins>)

```
    <execution>
      <id>npm-ci</id>
      <goals><goal>npm</goal></goals>
      <configuration><arguments>ci</arguments></configuration>
    </execution>
    <execution>
      <id>npm-build</id>
      <goals><goal>npm</goal></goals>
      <phase>generate-resources</phase>
      <configuration><arguments>run build</arguments></configuration>
    </execution>
  </executions>
</plugin>
```

Opção B: o Maven faz tudo (frontend-maven-plugin)

Terminal

```
aluno@pc:~/projeto/Back$ ./mvnw clean package -DskipTests
[INFO] --- frontend:1.15.1:install-node-and-npm (instalar-node) @ Back ---
[INFO] Installing node version v22.20.0
[INFO] --- frontend:1.15.1:npm (npm-ci) @ Back ---
[INFO] added 52 packages in 4s
[INFO] --- frontend:1.15.1:npm (npm-build) @ Back ---
[INFO] ✓ built in 702ms
[INFO] --- spring-boot:4.1.1:repackage (repackage) @ Back ---
[INFO] BUILD SUCCESS
aluno@pc:~/projeto/Back$ java -jar target/Back-0.0.1-SNAPSHOT.jar
Tomcat started on port 8080 (http)
```

Um comando gera o `.jar` com o back e o front. Abra `http://localhost:8080` e o React aparece.

Opção B: o Maven faz tudo (frontend-maven-plugin)

Observação

Combine com o `outDir` da opção A para que o build do Vite caia em `static/` antes do Maven empacotar. Outra forma comum é deixar o Vite gerar em `Front/dist` e usar o `maven-resources-plugin` para copiar `../Front/dist` para `target/classes/static` na fase `process-resources`.

04

Ajustes de segurança e deploy

Ajustes de segurança e deploy

- **Spring Security**: libere os arquivos do front, senão a página inicial pede login.

04

Ajustes de segurança e deploy

Aula 07 · FullStack

Ajustes de segurança e deploy

SecurityConfig.java (trecho)

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/", "/index.html", "/assets/**", "/favicon.ico", "/vite.svg").permitAll()
    .requestMatchers(HttpMethod.POST, "/api/login").permitAll()
    .requestMatchers("/api/**").authenticated()
    .anyRequest().permitAll())      // rotas do React Router caem no index.html
```

Ajustes de segurança e deploy

- **Dockerfile**: como o build agora precisa de Node, use uma etapa `node` para o front e outra `maven` para o back, copiando o `dist` para o `static` antes de empacotar (Deploy, aula 03 mostra a estrutura multi-stage).
- **Render**: o *Root Directory* passa a ser a **raiz** do repositório (o Dockerfile precisa enxergar `Front/` e `Back/`).

04

Ajustes de segurança e deploy

Aula 07 · FullStack

05

Aprofundando: um Dockerfile para o projeto unificado

Aprofundando: um Dockerfile para o projeto unificado

Dockerfile na RAIZ do repositório (três etapas)

```
# ETAPA 1: build do React
FROM node:22-alpine AS front
WORKDIR /front
COPY Front/package.json Front/package-lock.json ./
RUN npm ci
COPY Front/ .
RUN npm run build                       # gera /front/dist

# ETAPA 2: build do Spring, com o dist dentro do static
FROM maven:3.9-eclipse-temurin-25 AS back
WORKDIR /app
COPY Back/pom.xml .
RUN mvn -q -B dependency:go-offline
COPY Back/src ./src
COPY --from=front /front/dist ./src/main/resources/static
RUN mvn -q -B clean package -DskipTests

# ETAPA 3: execução
FROM eclipse-temurin:25-jre
WORKDIR /app
COPY --from=back /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "app.jar"]
```

Aprofundando: um Dockerfile para o projeto unificado

Neste formato o Vite usa o `outDir` padrão (`dist`) e é o Docker quem copia o resultado para o `static`. No Render, o *Root Directory* fica vazio (raiz) e o *Dockerfile Path* é `./Dockerfile`.

Onde fica o front	URL do front	URL da API	CORS
Vercel + Render	`app.vercel.app`	`api.onrender.com`	Necessário
Dentro do Spring	`api.onrender.com/`	`api.onrender.com/api`	Não precisa
nginx + Spring (compose)	`localhost:5173`	`localhost:8080`	Necessário (ou proxy no nginx)

Aprofundando: um Dockerfile para o projeto unificado

Alternativa: o nginx do front faz proxy da API (mesma origem)

```
location /api/ {
    proxy_pass http://back:8080;        # nome do serviço no compose
    proxy_set_header Host $host;
}
location / {
    try_files $uri $uri/ /index.html;
}
```

Cuidados com a versão unificada

Cache do index.html

Configure `Cache-Control: no-cache` para o `index.html`; os assets com hash podem ter cache longo.

Rotas da API

Tudo da API começa com `/api` para não colidir com as rotas do React Router.

Tamanho do jar

O build do front entra no jar; mantenha imagens grandes fora do `src/assets`.

Cold start

No Render Free, o site inteiro dorme junto com a API.

Aprofundando: um Dockerfile para o projeto unificado

Observação

Esta abordagem é a mesma usada por muitos projetos Spring Boot em empresas para painéis internos: um único artefato, uma única URL, sem configuração de CORS. Para sites públicos com muitos acessos, a CDN do Vercel costuma entregar o front mais rápido.

06

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Onde o Spring Boot serve arquivos estáticos?

A templates/

B src/main/resources/static/

C public/

D target/

Quiz · questão 1 de 5

Resposta

Onde o Spring Boot serve arquivos estáticos?

A templates/

✓ src/main/resources/static/

C public/

D target/

Por quê: Por isso o build do React pode ir para lá.

Pergunta

Qual opção do Vite manda o build para o back?

A server.port

B build.outDir

C plugins

D base

Resposta

Qual opção do Vite manda o build para o back?

A server.port

✓ build.outDir

C plugins

D base

Por quê: `outDir: "../Back/src/main/resources/static"`.

Quiz · questão 3 de 5

Pergunta

Para que serve o `SpaConfig`?

A CORS

B Devolver index.html para rotas do React (F5 funciona)

C Conectar ao banco

D Gerar JWT

Resposta

Para que serve o `SpaConfig`?

A CORS

✓ Devolver index.html para rotas do React (F5 funciona)

C Conectar ao banco

D Gerar JWT

Por quê: Arquivos existentes e a API continuam funcionando.

Pergunta

Com o front dentro do back, o CORS é necessário?

A Sim

B Não: front e API têm a mesma origem

C Só no Render

D Só no Windows

Resposta

Com o front dentro do back, o CORS é necessário?

A Sim

✓ Não: front e API têm a mesma origem

C Só no Render

D Só no Windows

Por quê: As URLs da API ficam relativas.

Pergunta

O que o frontend-maven-plugin faz?

A. Deploy no Vercel

B. Roda npm ci e npm run build durante o build do Maven

C. Testa o CORS

D. Cria o banco

Resposta

O que o frontend-maven-plugin faz?

A. Deploy no Vercel

✓ Roda npm ci e npm run build durante o build do Maven

C. Testa o CORS

D. Cria o banco

Por quê: `./mvnw package` gera um jar com tudo.

Fechamento

Resumo da aula

- ✓ O Spring serve `static/`; colocando o build do React lá, front e API ficam na mesma origem.
- ✓ Opção A: `build.outDir: "../Back/src/main/resources/static"` no `vite.config.js`.
- ✓ `SpaConfig` devolve o `index.html` para rotas que não são arquivo nem API (F5 funciona).
- ✓ Opção B: `frontend-maven-plugin` roda `npm ci` + `npm run build` no `./mvnw package`.
- ✓ Mesma origem: URL relativa, sem CORS; libere os estáticos no Spring Security.

Exercícios

1. Integre o Front do cardápio (aula 05) dentro do Back com a opção A e acesse tudo pela porta 8080.

2. Adicione React Router com duas rotas e prove que o F5 funciona graças ao `SpaConfig`.

3. Configure o `frontend-maven-plugin` e gere um `.jar` único; rode com `java -jar`.

4. Liste os prós e contras desta abordagem para um projeto que terá muitos acessos.

Para ir além

- github.com/eirslett/frontend-maven-plugin
- vite.dev/config/build-options#build-outdir

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 07?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web